

UNIFIED DECLARATIVE CROSS-PLATFORM FLEET MANAGEMENT SYSTEM

**A MINI PROJECT SYNOPSIS SUBMITTED
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF DEGREE OF**

**BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING**

SUBMITTED BY

Radhey Kalra (Roll Number: 2023A1R044)



SUBMITTED TO

Department of Computer Science and Engineering
Model Institute of Engineering and Technology (Autonomous)
Jammu, India
2026

CONTENTS OF SYNOPSIS

S. No	Topic	Page No
1.	Project Overview	3
2.	Problem Statement	3
3.	Objectives of the Project	3
4.	Proposed Solution / Methodology	4
5.	Key Technologies and Tools	5
6.	V.E.T.S Justification	6
7.	Expected Outcomes	7
8.	Implementation Timeline	8
9.	References	9

1. Project Overview

This project develops a unified declarative fleet management system for heterogeneous operating systems. The system belongs to the domain of Systems Engineering, Cloud Infrastructure, and Information Security. Contemporary enterprise environments operate mixed fleets of Linux, macOS, and Windows workstations that suffer from configuration drift and high management overhead. The proposed solution provides a central Go server that evaluates pure Nix Flakes to generate cryptographic target states for all client nodes. Native endpoint agents enforce these immutable states, verify cryptographic binary closures, and detect unauthorized drift. The target users are enterprise system administrators, site reliability engineers, and IT compliance officers who require verifiable infrastructure state across distributed endpoints.

2. Problem Statement

Modern enterprise organizations operate heterogeneous computing fleets containing Linux, macOS, and Windows machines. Current configuration management tools use imperative scripts that execute divergent commands and mutate local host state directly. These imperative updates frequently fail midway, leave systems in partially configured states, and produce silent configuration drift. Existing tools lack unified cryptographic verification for deployed system configurations and cannot execute atomic rollbacks when runtime health checks fail. Enterprise organizations must deploy separate, expensive vendor agents such as SCCM, Intune, Jamf, and Ansible on the same network. Operating separate management silos increases software license expenses and wastes wide-area network bandwidth during software updates. A clear technical gap exists for a single hermetic control plane that provides immutable state reconciliation and peer-to-peer binary distribution across all three operating systems.

3. Objectives of the Project

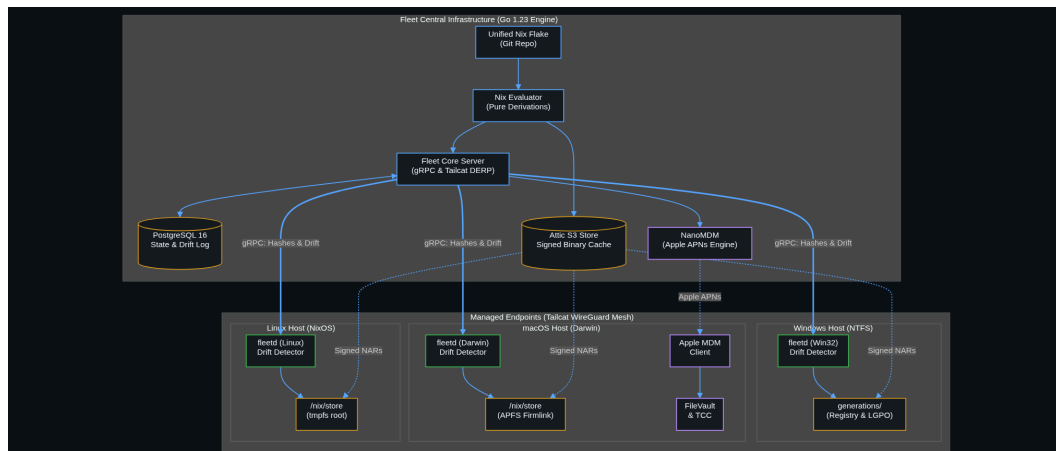
The objectives of this project are:

- **Design** a unified declarative schema using Nix Flakes that hermetically specifies system configurations, packages, registry settings, and security policies for Linux, macOS, and Windows.

- **Develop** a high-performance central control plane in Go 1.23 that evaluates declarative Flake inputs, dispatches target hashes over mTLS gRPC, and logs drift events in PostgreSQL 16.
- **Implement** lightweight native reconciliation agents for Linux, macOS, and Windows that continuously compare host state against target hashes and enforce atomic rollbacks upon failure.
- **Construct** an embedded peer-to-peer distribution layer using a Tailcat userspace WireGuard mesh and FastCDC content chunking to stream signed binary cache archives across local network peers.
- **Evaluate** system performance, configuration convergence time, WAN bandwidth reduction, and automatic drift detection latency across a multi-node testbed.

4. Proposed Solution / Methodology

The proposed solution establishes a declarative, pull-based architecture centered on cryptographic target states. Administrators define host profiles in a single central Git repository using modular Nix Flakes. The methodology divides the architecture into three technical tiers:



4.1 Central Control Plane

The central infrastructure hosts the Fleet Core Server, Nix Evaluator, PostgreSQL 16 database, Attic S3 binary cache, and an embedded NanoMDM service. The Nix Evaluator compiles declarative configurations into pure build derivations. The Attic cache compresses and stores signed Nix Archives (NARs) verified with Ed25519 cryptographic keys. The Fleet Core Server broadcasts desired SHA-256 target hashes to connected endpoints through outbound-only mTLS gRPC over HTTPS port 443. For macOS endpoints, the embedded NanoMDM service transmits Apple

Push Notification service (APNs) commands to configure native security profiles.

4.2 Native Endpoint Agents

Each target operating system executes a native, minimal reconciliation agent:

- **Linux Host (NixOS):** The `fleetd` agent mounts the root filesystem on an ephemeral `tmpfs` RAM disk while maintaining `/nix/store` in read-only mode. All unauthorized runtime mutations vanish upon system reboot, guaranteeing zero state drift.
- **macOS Host (Darwin):** The `fleetd` agent configures system packages through APFS synthetic firm links into `/nix/store`. Security baselines, including FileVault volume encryption and Transparency Consent and Control (TCC) profiles, are enforced via the native Apple MDM framework.
- **Windows Host (NTFS):** The `fleetd-windows` agent parses desired-state JSON specifications into timestamped generation bundles under `C:\ProgramData\Fleet\generations\<hash>\`. The agent natively compiles and applies Win32 registry hives, Local Group Policy Objects (LGPO), system services, and Winget packages without requiring UNIX emulation layers.

4.3 P2P Mesh Distribution and Automated Rollback

Endpoints form an encrypted peer-to-peer overlay network using a Tailcat userspace WireGuard mesh powered by gVisor netstack. When a new target hash is received, nodes discover local LAN peers via cryptographic discovery. Missing binary store chunks are retrieved from adjacent LAN workstations using FastCDC content chunking, reducing WAN gateway saturation by up to 90%. If an update fails health verification, the agent atomically resets the active generation pointer to generation $N - 1$, executing an immediate, deterministic rollback.

5. Key Technologies and Tools

The implementation utilizes the following core technologies:

- **Programming Languages:** Go 1.23 for the central control server and Windows agent runtime; Nix for hermetic configuration declarations; PowerShell and Win32 C bindings for Windows policy enforcement.

- **Communication and Protocols:** gRPC over HTTP/2 with mutual TLS (mTLS) authentication; Apple APNs for MDM command transport; WireGuard protocol for network overlay transport.
- **Networking and Relays:** Tailcat userspace network mesh with gVisor netstack; embedded DERP (Designated Encrypted Relay for Packets) relay on HTTPS port 443 for traversal across symmetric NAT firewalls.
- **Databases and Caches:** PostgreSQL 16 for machine inventory, target hashes, and drift audit logs; Attic binary cache backed by S3 object storage with Zstandard compression and FastCDC deduplication.
- **Operating System Subsystems:** Linux tmpfs and systemd; macOS APFS synthetic firmwares and Apple MDM framework; Windows Registry APIs, LGPO binary format, and Winget package manager.

6. V.E.T.S Justification

The project conforms to the evaluation criteria established in the V.E.T.S Framework:

V - Viability

The project is technically and operationally feasible within the allocated academic semester. The architecture builds on proven, production-grade open-source components, including Go 1.23, Nix, WireGuard, and PostgreSQL. The development team possesses strong background in systems programming, operating system internals, and distributed networks. A functional lab testbed running Linux (NixOS), macOS (Darwin), and Windows 11 virtualized environments is operational and verified.

E - Engineering Depth

The project requires significant systems engineering depth beyond simple software interfaces:

- **Cross-Platform State Compilation:** Designing a translation engine that maps unified Nix Flake declarations to native Windows Win32 registry hives, binary LGPO files, and macOS property lists.
- **Cryptographic Verification Pipeline:** Enforcing Ed25519 digital signature validation on all binary closures prior to disk activation.
- **Userspace Network Engineering:** Integrating a zero-TUN userspace WireGuard mesh with automated NAT hole-punching and HTTPS 443 DERP fallback relaying.

- **Atomic Reversal Engine:** Engineering continuous drift detection routines that compute host state divergence and perform instant atomic generation rollbacks within three seconds.

T - Trend Alignment

The project aligns with contemporary enterprise computing standards:

- **Infrastructure as Code (IaC):** Extends GitOps and declarative configuration management from cloud servers to end-user workstations.
- **Immutable Operating Systems:** Implements ephemeral root filesystems (tmpfs) and synthetic APFS firmlinks to eliminate persistent system corruption.
- **Zero Trust Architecture (ZTA):** Enforces continuous cryptographic identity verification, outbound-only mTLS connections, and peer-to-peer WireGuard transport without opening inbound perimeter firewall ports.

S - Social / Industrial Impact

- **Cost Optimization:** Eliminates recurring enterprise licensing fees for multiple fragmented management tools (Microsoft Intune, SCCM, Jamf Pro, and Ansible Tower).
- **Bandwidth Efficiency:** Reduces enterprise WAN bandwidth consumption by up to 90% via local LAN chunk swarming during large software distribution cycles.
- **Security Resilience:** Prevents persistent endpoint malware infection through ephemeral root reboots and continuous configuration drift detection.
- **Operational Simplicity:** Empowers IT departments to administer multi-platform computing environments through a unified, reproducible codebase.

7. Expected Outcomes

The project will deliver the following technical artifacts:

- A high-performance Fleet Core control server written in Go 1.23 with gRPC services, embedded DERP relay, and administrative audit logging.
- Native, low-overhead endpoint agents for Linux (fleetd), macOS (fleetd), and Windows (fleetd-windows).
- A production-grade unified Nix Flake repository template containing host modules for NixOS, nix-darwin, and Windows generation engines.

- A functional peer-to-peer binary distribution engine that streams verified closure chunks across local network peers.
- An automated drift detection and atomic rollback mechanism that detects unauthorized host modifications and restores verified state within three seconds.
- Comprehensive benchmark documentation evaluating configuration convergence time, WAN bandwidth savings, and rollback reliability.

8. Implementation Timeline

The project implementation follows a structured five-phase development schedule:

Phase	Activity
Phase 1 (Weeks 1–3)	Requirement analysis, declarative schema design, and local multi-OS lab testbed configuration.
Phase 2 (Weeks 4–7)	Fleet Core Server development in Go 1.23, PostgreSQL 16 schema design, and Nix Flake evaluator pipeline.
Phase 3 (Weeks 8–11)	Native endpoint agents development: Linux tmpfs root, macOS APFS/MDM, and Windows Win32/LGPO reconcilers.
Phase 4 (Weeks 12–14)	Tailcat WireGuard userspace mesh integration, DERP relay deployment, and FastCDC LAN P2P swarming.
Phase 5 (Weeks 15–16)	End-to-end integration testing, drift detection evaluation, performance benchmarking, and synopsis presentation.

9. References

1. E. Dolstra, “The Purely Functional Software Deployment Model,” Ph.D. dissertation, Department of Information and Computing Sciences, Utrecht University, Utrecht, Netherlands, 2006.
2. J. A. Donenfeld, “WireGuard: Next Generation Kernel Network Tunnel,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, 2017.
3. Apple Inc., “Mobile Device Management Protocol Reference,” *Apple Developer Documentation*, Cupertino, CA, USA, Tech. Rep., 2024.
4. Microsoft Corporation, “[MS-GPREG]: Group Policy: Registry Extension Encoding,” *Microsoft Open Specifications*, Redmond, WA, USA, Tech. Rep., 2024.
5. W. Xia, H. Jiang, D. Feng, F. Douglass, P. Shilane, M. Hua, G. Min, Y. Zhang, and B. Mao, “FastCDC: A Fast and Efficient Content-Defined Chunking Approach for Data Deduplication,” in *Proceedings of the 2016 USENIX Annual Technical Conference (USENIX ATC 16)*, Denver, CO, USA, 2016, pp. 101–114.
6. Google LLC, “gRPC: A High Performance, Open Source Universal RPC Framework,” *gRPC Project Documentation*, 2023. [Online]. Available: <https://grpc.io>
7. Tailscale Inc., “How Tailscale Works: An In-Depth Architecture Overview,” *Tailscale Engineering Documentation*, 2023. [Online]. Available: <https://tailscale.com/blog/how-tailscale-works>
8. PostgreSQL Global Development Group, “PostgreSQL 16.0 Documentation,” *The PostgreSQL Global Development Group*, 2024. [Online]. Available: <https://www.postgresql.org/docs/16/>
9. National Institute of Standards and Technology (NIST), “Zero Trust Architecture,” *NIST Special Publication 800-207*, Gaithersburg, MD, USA, Tech. Rep., Aug. 2020.
10. Aerospace, Security and Defence Industries Association of Europe (ASD), “Simplified Technical English: Specification ASD-STE100,” *ASD Standard*, Brussels, Belgium, Issue 8, Jan. 2021.